PAPER 3 — HANDOFF v12.0

Written 27 August 2026, end of session. Supersedes v11.0 where they disagree. Keep v11.0 for the grid-designer story (transposition, palm, throat) and v10.0 for Block 2/3 and the diagonal-blob finding; neither is repeated here in full.

0. WHAT THIS SESSION WAS

Two threads. First, **closing the loop**: the gripper now measures contact while it closes, which removed the last hand-tuned number from the pipeline and made a dead grasp visible as it happens. Second, **objects**: Berith's 23 meshes can now be loaded, and four cylinder widths have been validated end to end through that path.

Verify file freshness before editing. Grep for a signature:

File	Signature to grep for
main_gui.py	_obj_by_label
main_gui.py	_live_tactile_publisher / Live Tactile
sim/collect_from_config.py	read_contact_signal
sim/collect_from_config.py	placed by MEASUREMENT
viz/stitching.py	_load_dead_grasps
examples/probe_finger_throat.py	CAL_PATHS
examples/run_batch.py	NEW this session
Data/build_object_library.py	NEW this session

1. CONTACT-AWARE CLOSING

Until now every grasp closed to a **fixed close_rad** read from the calibration file: an angle picked by hand per diameter, with a D26 fallback for anything not in the file, and no check that contact happened at all. That is how a D60 sweep produced peak-4 maps for a whole run without anything noticing.

1.1 How the signal is read

The collector runs INSIDE Isaac, in the same process as the TSF-85 extension, so it finds the live Extension instance by walking the heap and reads `_last_pred` (s1) and `_sensor2.last_pred` (s2) directly. No streaming, no server, no ROS2 — and no edit to Berith's code, so his next model drop cannot break it.

Route	Verdict
Tail the CSV	Dead. logging_data.py flushes every 50 frames; a close ramp is 60 frames, so roughly one update per close.
Run the ONNX model ourselves	Rejected. We would have to reproduce his preprocessing exactly, and getting it subtly wrong yields plausible-but-wrong tactile numbers.
Read his live output	Chosen. Cannot be subtly wrong, and touches none of his files.

THE CATCH, and it is in the code. Inference is an async loop that pops the newest frame and CLEARS the queue, so predictions are not synchronised with physics and can go stale with no outward sign. Every read therefore carries its frame number, and if the frame never advances during a close the run prints `TACTILE WENT STALE` and says to treat the run as fixed-angle. Silence would be worse than failure here.

1.2 Two signals, and three wrong thresholds before they worked

Recorded because each wrong version looked reasonable and ran cleanly:

1. **Absolute deformation, target 1.0.** The pad reads max|sim-rest| ~= 1.15 AT REST, so the target was already exceeded before the fingers moved: it stopped after 1 frame of 60 at 0.0088 rad and the gripper visibly never closed. Same lesson SUBTRACT_BASELINE encodes for the tactile path.
2. **Raw max|sim-rest| minus a baseline.** Still wrong: that statistic is dominated by a RIGID-BODY offset, not by squash. On a firm D26 grasp (tactile peak 14339) closing moved s1 by +0.0285 and s2 by -0.0285 — equal and opposite, because the pads rotate in mirror directions. Contact was invisible underneath it.
3. **Dead-grasp floor of 0.05.** The units are stage units, so that was 50 mm. Absurd.

What works: fit the best rigid transform (Kabsch, SVD of the 3x3 covariance) mapping rest onto sim, apply it, and measure the residual. A rigid pad leaves nothing; a squashed one leaves the squash. Verified on synthetic data: pure rigid motion reads **0.000000** where the old statistic read 900, and dents of 1.0 and 0.2 read 0.95 and 0.19.

Signal	At rest	Firm grasp	Target now	Available
Pad indentation (mm)	0.000	1.3 – 1.5 mm	1.40 mm	yes
Tactile sum (counts)	~250	10 000 – 16 000	6000	yes

Both targets are TRIP POINTS, not final firmness: the pad keeps loading after the fingers stop. Measured on D26 — deformation target 1.0 mm settles at 1.31; tactile target 6000 settles at 11602, roughly double.

1.3 close_rad stays a hard ceiling

Contact-aware closing can only ever stop EARLIER than the calibrated angle, never squeeze harder. So the worst case of a bad signal is the behaviour we already had, not a crushed object.

1.4 Dead-grasp detection, and what it does downstream

Every grasp — in either closing mode — has its indentation read at the hold and written into execution_ledger.json, with a `dead_grasp` flag below 0.2 mm and a run-level `closing` summary. It never aborts: one bad point must not cost the other ninety-nine.

stitching.py now DROPS flagged grasps before the pose-outlier check, reporting the two reasons separately. Their cells stay unvisited, which the pair mask already handles. Left in, a dead grasp paints zeros indistinguishable from measured no-contact, and the network would learn the outline of our failures as a fact about the object. Runs predating the flag restitch identically.

Dead grasps are NOT retried, by design. The failure is geometric — the pose cannot reach contact — so a retry of the identical pose gives the identical result and only burns time in an unattended run.

2. CALIBRATION — THREE FILES, AND WHICH ONE IS IN FORCE

File	How made	Holds
pad_offset_calibration.json	hand-picked close_rad, 3–20 Aug	9 widths
..._fixed.json	same method, re-measured today	10 widths
..._contact.json	closed until the contact target	10 widths

The GUI picks which one is read, and it drives BOTH the grid designer and the run command. They must agree: the offsets differ by up to 0.73 mm between methods, and a mismatch puts every pad that far off with nothing to show it. Default is now **contact**.

"Both (compare)" mode runs ONE launch, ONE descent, TWO closes at the same pose — pt00 fixed, pt01 contact-aware — so the only difference between the two files is the closing itself. Comparing against the hand-tuned file instead would mix method change with three weeks of drift.

2.1 What the comparison showed

	fixed	contact-aware
peak spread across 9 diameters	52.6%	30.1%
indentation spread	35.3%	26.0%
D26 close_rad	0.557	0.5265
D26 TOOL_OFFSET_Z	0.15656	0.15631
repeatability across launches	—	0.5266 / 0.5265 rad, offset identical to 5 dp

So contact-aware roughly halves the variation in how hard the sensor is driven across a 6x diameter range — the thing you would want constant when one network must read all of them. And fixed-today matched the hand-tuned file to **0.02 mm**, so nothing drifted in three weeks.

> ***A guard that fired correctly:*** *the contact-aware D18 grasp was REFUSED because its tactile peak (7513) fell under the sane band 8148–32593. Contact-aware deliberately squeezes lighter, so on small diameters the target has to be raised — 1.0 mm was not enough, 1.4 was. If this keeps happening the band needs rethinking for contact mode rather than the target being nudged each time.*

3. OBJECTS — BERITH'S 23 MESHES

Sizes: XS 9, S 18, M 26, G 40, XG 60, XXG 80 mm across; bodies 100 mm (spheres 60/90). **_H means the object lies down** — long axis on Y, not Z. Four shapes: cylinder, cuboid, cube, sphere. Stage units are consistent at 1.0. The "spheres" are 40x40x90 and 65x65x90, i.e. capsules — check one visually before assuming Paper 1's spherical rule applies.

3.1 The pipeline

Step	What	When
measure_objects.py	bbox of every USD -> object_sizes.json	once, unless the meshes change
calibrate (GUI)	close_rad + TOOL_OFFSET_Z for that WIDTH	once per new width
probe_finger_throat.py	how deep the throat lets each width go	after any NEW width
build_object_library.py	combines all three -> object_library.json	after the probe

Both of the last two are now buttons on the Calibrate tab. The library is what the object dropdown reads to decide "runnable" and what pad_dz to fill in.

The probe reads close_rad per width, so it can only measure widths that are already calibrated — that is why the order matters. It now merges all THREE calibration files; reading only the main file meant a width calibrated into _fixed/_contact silently got no throat entry, and the library then said "no throat entry for width 18.0 mm" with nothing obviously wrong anywhere.

3.2 Three placement bugs, all of which looked normal in the logs

1. **Nested rigid bodies.** Object_02 already IS the rigid body; Berith's USDs bring their own, giving a body inside a body, which PhysX does not define. The object fell, tipped over and slid before the fingers reached it. RemoveAPI alone does not override an API applied in a REFERENCED layer, so physics:rigidBodyEnabled is set False as well,

and the result is VERIFIED and printed.

2. **Cleared transform.** Adding the placement translate by clearing the referenced prim's xformOpOrder also threw away its authored orientation — Cylinder_M came up lying on its side. The translate now lives on a wrapper Xform we own, with the USD referenced into a child.
3. **Base versus centre.** The scene's convention is that Object_02's origin is the object's BASE — the authored unit cylinder is shifted up by L/2 for exactly that reason. Berith's meshes are already modelled on their base, so "re-centring" them pushed each one 50 mm down through the floor. Placement is now MEASURED: read the geometry's own bounds, put its base at z = 0.

And a pre-existing bug the new objects exposed. Object_02's position was INHERITED from the scene, which authored it as the base of a 140 mm rod, so the object's CENTRE drifted with length: a 100 mm body landed 20 mm low (measured 1032.2 against 1052.2). The same would have hit a scaled cylinder at any length but 140. The collector now measures the geometry's world bounds and shifts it so the centre lands on OBJ_CENTER; at L = 140 the correction is **0.0 mm**, so every run made before today is unchanged.

Separately, the GUI's "stand on table" (OBJ_BASE_Z = 982.2) was not firing on a library pick, so a 100 mm object floated 17 mm above the table. Every pick now sets obj_z = 982.2 + L/2.

3.3 Validated so far

Object	Result
Cylinder_S 18 x 100	calibrated both methods; grid runs
Cylinder_M 26 x 100	1.43 mm indentation, 0 dead
Cylinder_G 40 x 100	25 points, 0 dead, mean 1.29 mm
Cylinder_XG 60 x 100	single point, firm

Four consecutive widths end to end, so the cylinder route is done. Remaining widths (9, 12, 30, 50, 65, 80) are repetition of a proven procedure.

> ***Watch on D40:*** *indentation ranged 0.31 to 1.76 mm across 25 points — a 5.7x spread, far wider than the +-0.1 mm seen within a calibration. All well above the 0.2 mm floor, so nothing is wrong, but if the edge columns are systematically light it will show as a gradient in the stitched map.*

4. NEW TOOLS

4.1 Batch runner (examples/run_batch.py)

A queue of SESSION FOLDERS the GUI already wrote, run unattended. It invents no format: every item has already passed design_grid's throat, palm, canvas, band and per-point gripper checks before it can be queued. Written from the Batch tab; run in a terminal, so closing the GUI does not kill a 20-hour batch.

- **One process per run.** A crash in run 57 costs one run, not the queue. ~40 s startup against 30–40 min runs is about 4%.
- **Resume.** batch_state.json marks each entry pending/done/failed; re-running skips what finished. --status, --retry-failed.
- **A clean exit is not treated as success.** It reads the ledger, and a run where EVERY grasp was dead is marked FAILED even though the process exited 0 — the D60 jam scenario.
- **Collect only.** No stitching, pairs, blob or heatmaps: all are re-runnable, and each is another way for the queue to die at 3am.

Tested with a fake collector: success, missing config, all-dead caught despite exit 0, resume, and --status.

4.2 Live tactile tab

Both pads, live, during any run — single or batch, headless or not. The GUI is a SEPARATE PROCESS from Isaac and can never read _last_pred, so the collector publishes to Data/live_tactile.json at 20 Hz from a DAEMON THREAD (not a hook in the stepping code: world.step() is called from a dozen places and patching each would leave gaps exactly where the interesting things happen). Atomic writes, and every exception swallowed — a viewer must never be able to kill a run.

Drawn to match the real rig's Qt app (graphics.cpp): colour ramp B-b-c-y-r-R at 0/.17/.25/.35/.55/.85, a 1-cell zero pad before upsampling, view azimuth 250. Upsampling is Catmull-Rom cubic at x8 rather than the Qt app's linear x4. Axes in mm (33 x 47.6 including the pad ring), optional taxel grid, zero-baseline button, record to CSV.

> ***The scales are NOT comparable with the real rig*** *— that sensor has hysteresis and its own baseline, the CNN has neither. Same colours, same shape, different numbers. The figure says so.*

4.3 Smaller changes

- **log_mesh switch** on both Collection and Batch tabs. The per-frame mesh CSVs are ~98 MB per grasp (6.2 GB for a 63-point run) and NOTHING downstream reads them. Off saves ~59 GB over a 20-hour queue. Verified safe: tactile CSVs still written, heatmaps and stitching still work. Recorded in the ledger so a folder without them says so deliberately.
- **Ground plane fallback.** add_default_ground_plane() downloads from Nvidia's S3 bucket; with no route it blocks ~5 minutes then raises, killing the run before the scene exists — an empty Isaac window and a stack trace naming the ground plane, not the network. A local 50 m static collider is authored instead. cuRobo plans against the table slab, not this.
- **Ascent contact zone.** plan_stitched_z gained contact_at; the three descent callers are bit-identical, only the ascent passes "start". Fixes "Start or End state in collision" on the final point, which only appeared once the gripper entered the collision model.
- **Calibrate tab scrolls**, and sizes itself to its widgets.
- **Object dropdown**: grouped by shape, smallest first, upright before lying, dimensions in the label. Un-runnable objects are SELECTABLE — refusing them blocked the only route that could make them runnable — and apply size and USD but not pad_dz.

5. KNOWN-BROKEN / NOT YET DONE

Item	Severity	Note
Diagonal blob artifact	High	Berith has confirmed it is real and is working on it. The four live captures at 0/20/45/90 deg are the clearest evidence yet: 0 gives a clean vertical ridge and 90 a clean horizontal one, while 20 and 45 collapse into broad blobs — 45 deg has the HIGHEST sum (12195) with the LOWEST peak (999), the signature of energy smeared instead of concentrated. **Collect 0 and 90 deg only.**

Item	Severity	Note
Cuboid contact band	High	design_grid's across limit is sqrt(2.4*(D-2.4)) — a CYLINDER formula. A cuboid has a flat face: the pad touches all of it, so the limit is the face width, not a band. The library already reports Cuboid_M as READY, so a cuboid grid would be designed on the wrong rule and nothing would complain.
Cuboid yaw is undefined	Medium	A cylinder looks the same from every direction; a cuboid has faces AND edges, and a corner grasp is a different tactile signature. The scene needs a defined yaw and the config must record it.
Plots draw a cylinder	Low	Preview, calibration plot, stitched map and blob overlay all draw a round outline. Sizes are correct; the shape is not.
100 mm vs 140 mm bodies	Medium	The six earliest sweeps are 140 mm scaled cylinders; the new objects are 100 mm. Not the same object — do not pool them without saying so.
validation.py rolled pads	Low	_sample_canvas still assumes an axis-aligned footprint. Correct on flat runs.
GSR saturated	Low	No dynamic range; do not quote it.

6. VERIFIED CONSTANTS

Constant	Value	Provenance
PALM_DROP_MM	86.69	measured: 10.79 + 75.9
THROAT_MARGIN_MM	1.0	bracketed: dead -0.06, D26 lowest +1.88, calibration pose +3.13
DEAD_GRASP_FLOOR	0.0002 m	0.2 mm; rest 0.000, firm 1.3–1.5
CONTACT_TARGET default	0.0014 m	1.4 mm; 1.0 fell under the calibration sanity band
OBJ_BASE_Z	982.2 mm	table-top convention; obj_z = base + L/2
grid step default	6.0 mm	unchanged
PAD_W, PAD_H	22.0, 37.0 mm	unchanged
row 0	pad's physical BOTTOM	proven 2026-07-22

Throat table (shallowest depth past the flange a rod of that width may occupy):

D (mm)	10	13	18	20	26	32	40	42	45	52	60
min depth	13	14	*	102	102	102	*	104	104	110	112

** measured this session; re-read finger_throat.json for the current values rather than trusting this table.*

7. FILE INVENTORY

File	State
main_gui.py	object dropdown + library, live tactile tab, batch tab, calibration-source selector, contact-aware controls, post-calibration buttons, log_mesh
sim/collect_from_config.py	contact-aware close, live publisher, external USD loading, placement by measurement, ground-plane fallback, ascent contact zone, log_mesh
viz/stitching.py	drops dead grasps
examples/probe_finger_throat.py	merges all three calibration files
examples/run_batch.py	**NEW**
examples/measure_objects.py	**NEW**
Data/build_object_library.py	**NEW**
Data/object_library.json	**NEW** — 23 objects, 10 runnable
Data/finger_throat.json	regenerated; now includes 18 and 40
Data/pad_offset_calibration{,_fixed,_contact}.json	3 files, 9/10/10 widths

8. WHAT TO DO NEXT

1. **Cuboid contact band and yaw** — the only thing blocking a second shape class, and the thing that decides whether cuboid data means anything.
2. **Calibrate the remaining widths** (9, 12, 30, 50, 65, 80), then throat probe, then rebuild library. Proven procedure, just repetition.
3. **Then the batch runner earns its keep**: queue the cylinders at 0 and 90 deg with log_mesh off and collect at scale.
4. **Calcul Quebec in parallel** — access has latency that cannot be compressed later.

Working preferences, carried forward: one verifiable step at a time; measure, don't guess; mm; complete files, never diffs; commands on one line. **Kourosh's visual read of the simulation overrides diagnostics when they conflict — this has been correct every time, including twice this session** (the wobbling object, and the pad sitting off the top of a short body).

> *A pattern worth keeping from this session: of the roughly ten faults found, the ones that cost the most time were all cases where a WRONG NUMBER LOOKED NORMAL — a stale label, an inherited position, a statistic measuring the wrong thing. Every new check added since is expected to print what it measured, not just its verdict.*